

CPSC 375 High-Performance Computing

Lecture 13

Thread Synchronization

Mutex

Pthreads Interface

- Creating and *reaping* threads
 - `pthread_create()`
 - `pthread_join()`
- Determining thread ID
 - `pthread_self()`
- Terminating threads
 - `pthread_cancel()`
 - `pthread_exit()`
 - `exit()` [terminates all threads]
- Synchronizing access to shared variables
 - `pthread_mutex_init`
 - `pthread_mutex_[un]lock`
 - `pthread_cond_init`
 - `pthread_cond_[timed]wait`

Thread Synchronization

- Synchronization is the coordination of thread execution to ensure correctness.
- It addresses the *race condition* where multiple threads access shared memory.
- Without synchronization, the order of memory access is non-deterministic.
- The goal is to enforce *mutual exclusion* for critical sections.
- It also handles *condition synchronization*—waiting for data to reach a certain state.
- *Pthreads* provides three primary tools: mutexes, condition variables, and `pthread_once`.

Mutex

- Most basic synchronization tool in the Pthreads for ***mutual exclusion***.
- Acts as a lock ensuring only one thread can access a resource.
- When a thread locks a mutex, it becomes the owner of that mutex.
- Other threads attempting to lock the same mutex are *blocked*.
- Essential for protecting global variables in the shared data segment.
- Prevent the *load-update-store* race condition.
- Should be held for the shortest time possible to maintain performance.
- Process-wide—can be accessed by any peer thread in the process.
- A mutex must be unlocked only by the thread that originally locked it.

Initializing a Mutex

- Represented by the `pthread_mutex_t` data type.
- A mutex must be initialized exactly once before use.
- Two ways:

- Static for global mutexes

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER
```

- Dynamic for mutexes on the heap or stack

```
int pthread_mutex_init(pthread_mutex_t *mutex,  
                        const pthread_mutexattr_t  
*attr);
```

- `attr` allows for advanced settings but is usually `NULL`.
- Dynamic initialization is required when a mutex is part of a dynamically allocated structure.

Mutex Operations

- Locking

```
pthread_mutex_lock(&mutex);
```

- If the mutex is already locked, the calling thread blocks until it is available.

- Unlocking

```
pthread_mutex_unlock(&mutex);
```

- Releases the mutex and allows one blocked thread to wake up and acquire it.

- Trying a lock

```
pthread_mutex_trylock(&mutex);
```

- Returns `EBUSY` immediately if the mutex is locked, instead of blocking.
- Used to avoid deadlocks

- All three functions return 0 on success or an error code on failure.
- **Important!** Always check return codes to handle resource exhaustion or logic errors.

Destroying Mutexes

- A dynamic mutex should be destroyed when no longer needed:

```
pthread_mutex_destroy(&mutex);
```

- You cannot destroy a mutex that is currently locked.
- You cannot destroy a mutex that other threads are currently blocked on.
- Attempting to use a destroyed mutex results in undefined behavior.
- Destroying a static mutex is usually unnecessary but legal.
- Proper destruction is vital to prevent memory leaks in long-running HPC applications.

Mutexes and Data Structures

- High-performance programs often associate one mutex with a specific data structure, e.g., linked lists.
- With *fine-grained* locking:
 - head node is protected by a mutex.
 - allows threads to work on different parts simultaneously.
- With *coarse-grained* locking
 - One mutex protects the entire list.
 - Easier to implement, but leads to high lock contention.
- Contention causes threads to spend more time waiting than doing math.
- Fine-grained locking increases complexity and the risk of deadlocks.

Error Handling with Mutexes

- *Pthreads* functions do not set `errno`; they return the error code directly.
- Common error codes:
 - `EINVAL`: invalid mutex
 - `EDEADLK`: deadlock detected
 - `EPERM`: a thread tries to unlock a mutex it doesn't own
 - `EBUSY`: by `trylock` if the mutex is already held.
- Silent failures in synchronization are the primary cause of nonrepeatable errors in using threads.
- Debugging these requires analyzing the return values in real-time.
 - Employ tools like `valgrind` to detect memory errors and thread synchronization issues.

Performance Considerations

- Locking a mutex takes approximately 50–100 CPU cycles on modern hardware.
- If the mutex is already held, the cost jumps to thousands of cycles for context switching.
- High contention can make a threaded program slower than a sequential one.
- So, minimize the time spent in a critical section.
- Do not perform I/O (like `printf`) while holding a mutex unless necessary.
- Compute partial results in local variables on the stack before locking the global sum.



Demo II

- **Copy** pth_add.c:
`$ cp /home/pyoon/cpsc375/pth_add.c .`
- **Compile:**
`$ gcc -o pth_add pth_add.c -lpthread`
- **Run to add 1~1,000,000 using 8 threads:**
`$ ./pth_add 1000000 8`
- **Compare running times with various numbers of threads:**
`$ time ./pth_add 1000000 10`

Demo II with Mutexes

- Declare a global `sum` and a `mutex`.
- Modify the code so that each thread updates the global sum using the following two approaches:
 - **Per-addition:** Each thread acquires the mutex to update the global sum every time it adds an individual number.
 - **Per-thread:** Each thread computes its own local partial sum and updates the global sum only once, after its computation is complete.
- Measure the execution time for both cases and compare them against the unsynchronized version.